Universidad Tecnológica de Puebla
División de Tecnologías de la Información

Subproducto No. 6: Notificaciones

Notificaciones push en Rodando UTP

Avisos que llegan a tiempo, incluso con la aplicación cerrada

Fortino Romero Mantilla

Desarrollo de Software Multiplataforma

10 A – Aplicaciones Web Progresivas

Mtra. Xóchitl Mendoza Píscil

Periodo Septiembre–Diciembre 2026

28 de octubre de 2026

Índice

1. Introducción
2. ¿Qué son las notificaciones push?
3. Propósito
4. Características de las notificaciones push
    4.1 Ciclo de funcionamiento
    4.2 Entrega en segundo plano
    4.3 Contenido cifrado y servidor identificado
    4.4 Integración con la aplicación web progresiva
5. Ventajas
6. Aplicaciones prácticas
7. Componentes y tecnologías
    7.1 Piezas del sistema
    7.2 Requisitos y permisos
8. Ejemplo desarrollado
    8.1 Alcance y arquitectura
    8.2 Interfaz del laboratorio
    8.3 Flujo de extremo a extremo
    8.4 Resultados de la prueba
    8.5 Configuración y seguridad
9. Conclusiones
10. Referencias
Apéndice A. Correspondencia con los requisitos
Apéndice B. Enlaces de la entrega

1. Introducción

Rodando UTP es una aplicación web progresiva (PWA) para compartir raites entre estudiantes de la Universidad Tecnológica de Puebla. La aplicación ya permite consultar rutas, apartar un lugar, calcular el gasto de gasolina y seguir el recorrido del auto. Sin embargo, la información solo aparece cuando el usuario abre la aplicación: si el conductor cancela, si el auto ya llegó o si falta poco para la salida, el aviso depende de que alguien revise la pantalla o de un mensaje por otro medio.

Las notificaciones push resuelven ese hueco. Permiten que el servidor entregue un mensaje al navegador aunque la pestaña esté cerrada, y que el service worker lo muestre como un aviso del sistema. Este reporte explica qué son, para qué sirven, cómo funcionan y qué piezas necesitan; después presenta un ejemplo funcional dentro de Rodando UTP que envía y recibe una notificación real, documentado con capturas de la interfaz y con los resultados de la prueba.

El trabajo se organiza en tres partes. La primera define el concepto y sus características; la segunda describe ventajas, aplicaciones y componentes técnicos; la tercera documenta el laboratorio implementado, su arquitectura, su operación y las verificaciones realizadas.

2. ¿Qué son las notificaciones push?

Una notificación push es un mensaje que un servidor envía a un navegador o a una aplicación sin que el usuario haya solicitado la información en ese momento. En la web, la Push API permite que un service worker reciba esos mensajes del servidor incluso cuando la aplicación no está abierta o está en segundo plano (MDN Web Docs, s. f.-b). La Notifications API se encarga de mostrarlos con el aspecto y el comportamiento que el sistema operativo define para las notificaciones (MDN Web Docs, s. f.-a).

No deben confundirse con los avisos internos de la interfaz. Un mensaje emergente dentro de la página, como los que Rodando UTP muestra al guardar un raite, solo existe mientras la pestaña está visible. Una notificación push viaja por un servicio externo, despierta al service worker y puede aparecer en la bandeja del sistema con la aplicación cerrada.

Tampoco es una notificación local programada por el propio navegador: en la push el origen del mensaje es el servidor de la aplicación.

El estándar separa dos responsabilidades. La Push API transporta el mensaje y la Notifications API lo presenta. Entre ambas media un servicio push del navegador —por ejemplo, el servicio de Firebase Cloud Messaging en Chrome o el de Apple en Safari— que mantiene una conexión permanente con el dispositivo y entrega el mensaje cuando corresponde (Thomson et al., 2016).

3. Propósito

El propósito de una notificación push es informar a tiempo, sin obligar al usuario a vigilar la aplicación. Se usa cuando el valor del mensaje depende de su oportunidad: una confirmación que llega tarde pierde sentido, y un aviso de cancelación que no se ve puede dejar a una persona esperando en la calle. La notificación traslada la iniciativa del usuario a la aplicación: en lugar de preguntar «¿ya llegó?», el sistema avisa cuando ocurre.

En Rodando UTP ese propósito se concreta en cuatro momentos: confirmar que una reservación fue aceptada, avisar que el conductor va en camino, recordar la salida con anticipación y comunicar una cancelación o un cambio de horario. Los tres primeros ya forman parte del flujo de la aplicación; el cuarto se apoya en la misma infraestructura y puede activarse con el envío administrativo existente.

La intención no es saturar. Una notificación compite por la atención del usuario y, si se usa sin criterio, se percibe como molestia y termina desactivada. Por eso el diseño del ejemplo incluye un permiso explícito, un envío únicamente a las suscripciones de la propia cuenta y una bitácora que hace visible qué ocurrió en cada paso.

4. Características de las notificaciones push

Las push combinan entrega asíncrona, ejecución en segundo plano y un modelo de permisos estricto. Esas tres propiedades explican tanto su utilidad como sus límites.

4.1 Ciclo de funcionamiento

El ciclo tiene cuatro etapas. Primero el usuario concede el permiso de notificaciones. Después el navegador crea una suscripción con un endpoint único y dos valores criptográficos, p256dh y auth, que el servidor guarda asociados a la cuenta (MDN Web Docs, s. f.-c). Cuando la aplicación necesita avisar algo, el servidor firma la petición con su llave VAPID, cifra el contenido y lo envía al endpoint. Por último, el servicio push entrega el mensaje al navegador, el service worker lo recibe y llama a showNotification para mostrarlo (web.dev, s. f.).

La suscripción es estable, pero no eterna. Si el usuario borra los datos del navegador, desinstala la aplicación o el servicio push invalida el endpoint, el envío devuelve un error 404 o 410. El servidor debe interpretar esa respuesta como una suscripción caducada y eliminarla para no acumular registros inútiles. Rodando UTP ya aplica esa limpieza en el servicio de envío.

4.2 Entrega en segundo plano

La ventaja técnica más importante es que el mensaje no necesita una pestaña abierta. El service worker es un script que el navegador mantiene registrado y que puede despertar para atender un evento push aunque la aplicación esté cerrada. Esa capacidad es la que convierte a la PWA en un canal de avisos comparable al de una aplicación nativa, sin obligar a instalar una tienda de aplicaciones.

El alcance depende de la plataforma. En Android y en los navegadores de escritorio el soporte es amplio. En iPhone y iPad, la web push requiere iOS o iPadOS 16.4 o posterior y que la aplicación se haya agregado a la pantalla de inicio; si se usa solo dentro de Safari, la suscripción no está disponible (WebKit, 2023). El diseño debe prever esa diferencia y ofrecer una alternativa cuando la push no sea posible.

4.3 Contenido cifrado y servidor identificado

El contenido viaja cifrado de extremo a extremo entre el servidor y el navegador. La especificación RFC 8291 define el esquema de cifrado del payload con las llaves de la suscripción, de modo que el servicio push que actúa como intermediario no puede leer el mensaje (Thomson y Beverloo, 2017a). El servidor, a su vez, se identifica con VAPID: un par de llaves y un token firmado que demuestra que la aplicación es quien envía y no un tercero que conoce el endpoint (Thomson y Beverloo, 2017b).

Para el proyecto, esas propiedades implican dos reglas. La llave privada VAPID vive en el entorno del servidor y nunca se incluye en el repositorio ni en el cliente; la llave pública sí se entrega al navegador para crear la suscripción. Además, el endpoint de la suscripción se trata como un dato sensible, porque quien lo posea junto con las llaves podría intentar enviar mensajes; por eso la base de datos no expone esas filas en las respuestas públicas de la API.

4.4 Integración con la aplicación web progresiva

La push se apoya en la infraestructura que la PWA ya tiene: un manifiesto que declara la identidad de la aplicación, un service worker registrado, HTTPS y una interfaz que puede mostrar el estado del permiso. No requiere empaquetar la aplicación para una tienda ni mantener dos bases de código. El mismo service worker que guarda recursos para el modo sin conexión atiende los eventos push y de clic en la notificación.

Esa integración también impone límites. La push necesita conexión a internet para llegar; no sustituye la caché. Lo que sí puede hacer es mostrar un aviso con información que el service worker ya conoce, y al tocarlo abrir la sección correspondiente de la aplicación. En el ejemplo, el clic lleva a la página del laboratorio y las pestañas abiertas reciben un mensaje para actualizar su bitácora sin recargar.

5. Ventajas

Frente a otros canales, la push ofrece inmediatez sin exigir datos personales adicionales. El usuario no entrega su correo ni su teléfono: autoriza al navegador y recibe el aviso en el dispositivo donde instaló la aplicación. Para el proyecto, eso reduce la fricción y evita construir una base de contactos.

- Oportunidad: el mensaje llega cuando ocurre el hecho, no cuando el usuario recuerda abrir la aplicación.
- Cobertura: funciona con la pestaña cerrada y con la aplicación en segundo plano, siempre que el sistema operativo lo permita.
- Costo: no hay tarifa por mensaje con el servicio push del navegador; el gasto se limita al servidor que ya atiende la API.
- Control: el permiso se puede revocar en cualquier momento y la aplicación debe respetar la decisión sin insistir con ventanas emergentes.
- Integración: reutiliza el service worker, el manifiesto y la sesión existentes; no obliga a mantener una aplicación nativa aparte.
- Medición: el servidor sabe cuántas suscripciones aceptó el envío y puede depurar las caducadas, lo que permite evaluar la entrega.

La Tabla 1 contrasta la push con los dos canales que suelen usarse como respaldo. La comparación no pretende declarar un ganador, sino ubicar cuándo conviene cada uno.

Tabla 1

Canales de aviso y su comportamiento en Rodando UTP

Canal	Llega con la app cerrada	Dato personal	Uso adecuado
Notificación push	Sí, si el sistema lo permite	No	Confirmaciones, llegada y recordatorios
Aviso dentro de la app	No	No	Respuesta inmediata a una acción del usuario
Correo o mensaje SMS	Sí	Correo o teléfono	Respaldo, recuperación de cuenta o soporte

La lectura de la tabla es directa: la push sirve para lo urgente y operativo; el aviso interno, para confirmar lo que el usuario acaba de hacer; el correo o el SMS, para lo que debe quedar fuera de la aplicación o alcanzar a quien no la usa.

6. Aplicaciones prácticas

En Rodando UTP la aplicación más valiosa es la coordinación del viaje. El pasajero reserva con días de anticipación, pero la decisión de salir de casa ocurre minutos antes. Una push con el estado del auto o con el recordatorio de salida reduce la espera en la calle y las

cancelaciones por impuntualidad. El conductor, por su parte, puede recibir el aviso de que un pasajero ya lo espera en la parada.

- Confirmación de reservación: avisar que el conductor aceptó y que el lugar quedó apartado.
- Conductor en camino: informar que el viaje inició y que el auto se acerca al punto de encuentro.
- Recordatorio de salida: avisar unos minutos antes de la hora publicada.
- Cambio o cancelación: comunicar que el raite se pausó, se llenó o cambió de horario.
- Aviso operativo: informar mantenimiento, nuevas rutas o ajustes del servicio a los usuarios suscritos.

Fuera del proyecto, el mismo patrón aparece en comercio electrónico (confirmación de pedido y entrega), educación (tareas, calificaciones y recordatorios de clase), salud (citas y resultados), transporte (llegada del vehículo) y medios (noticias de última hora). La condición común es que el mensaje pierde valor si se lee tarde.

También hay aplicaciones que no conviene cubrir con push. La publicidad constante, las encuestas o los contenidos que el usuario puede consultar cuando quiera pertenecen a otro canal. La pregunta de diseño no es «¿podemos avisar?», sino «¿el usuario necesita saberlo ahora?». Si la respuesta es no, el aviso interno o el correo son suficientes.

7. Componentes y tecnologías

Un sistema de notificaciones push se divide en tres capas: el cliente que se suscribe y muestra, el servidor que decide y envía, y los protocolos que garantizan identidad y cifrado. La Tabla 2 resume las piezas del ejemplo y la función de cada una.

Tabla 2

Componentes y tecnologías del ejemplo

Capa	Componente	Función	Tecnología
Cliente	Permiso y suscripción	Solicita autorización y crea la suscripción	Notifications API y Push API
Cliente	Recepción en segundo plano	Despierta con el mensaje y muestra el aviso	Service worker
Cliente	Interfaz del laboratorio	Explica el flujo y registra cada evento	HTML, CSS y JavaScript
Servidor	API de suscripción	Guarda endpoint, p256dh y auth ligados a la cuenta	Node.js, Express y PostgreSQL
Servidor	Envío firmado	Cifra el payload y lo entrega al servicio push	web-push y llaves VAPID
Protocolo	Entrega HTTP	Transporta el mensaje hasta el navegador	RFC 8030
Protocolo	Cifrado del contenido	Impide que el intermediario lea el mensaje	RFC 8291
Protocolo	Identidad del servidor	Demuestra quién envía la notificación	RFC 8292 (VAPID)
Red	Servicio push	Mantiene la conexión con el dispositivo y encola el aviso	Servicio del navegador

7.1 Piezas del sistema

En el cliente, la Notifications API expone el permiso y la forma de mostrar el aviso; la Push API crea la suscripción y entrega los eventos al service worker. El service worker es el punto de unión: recibe el evento push, decide qué mostrar y atiende el clic para abrir la sección correcta. Sin él no hay push en la web, porque la página puede estar cerrada.

En el servidor, la API de Rodando UTP ya contaba con las rutas de suscripción y de envío administrativo. El ejemplo agrega una ruta de prueba que exige sesión, busca las suscripciones del usuario y devuelve cuántos envíos aceptó el servicio push. La librería web-push construye la petición, firma con VAPID y cifra el contenido; la base de datos conserva la suscripción y la limpia cuando el servicio responde 404 o 410.

Los protocolos completan el cuadro. El RFC 8030 define la entrega de eventos por HTTP; el RFC 8291, el cifrado del payload; el RFC 8292, la identificación del servidor con VAPID. No es necesario implementar esos protocolos a mano: las bibliotecas del servidor y el navegador los resuelven, pero conocerlos explica por qué se necesitan HTTPS, un par de llaves y una suscripción por dispositivo.

7.2 Requisitos y permisos

Para que el flujo funcione deben cumplirse cinco condiciones. La primera es un contexto seguro: HTTPS o localhost, porque el navegador no expone la push en conexiones sin cifrar. La segunda es un service worker registrado en el ámbito de la aplicación. La tercera es el permiso explícito del usuario; si lo niega, la aplicación no puede volver a mostrar el cuadro de diálogo por sí sola y debe indicar cómo cambiarlo en la configuración del navegador. La cuarta es un par de llaves VAPID configurado en el servidor. La quinta es almacenar la suscripción y asociarla a una cuenta para saber a quién enviar.

El momento del permiso importa. Pedirlo al cargar la página, sin contexto, produce rechazos. Conviene explicar el beneficio y solicitarlo cuando el usuario realiza una acción relacionada, por ejemplo al activar los avisos de un raite. En el ejemplo, la persona pulsa «Activar notificaciones» y solo entonces el navegador muestra su diálogo.

8. Ejemplo desarrollado

El ejemplo se implementó dentro de la misma PWA, no como una maqueta aislada. Agrega una página de laboratorio, una ruta de prueba en la API y dos ajustes al service worker: el aviso a las pestañas abiertas y el uso de la URL enviada por el servidor al tocar la notificación. La página se enlaza desde el pie del sitio y desde un atajo del manifiesto.

8.1 Alcance y arquitectura

La página se llama notificaciones.html y muestra el ciclo completo: estado del soporte, permiso, suscripción, envío y bitácora. La ruta POST /api/push/prueba exige sesión, recupera las suscripciones del usuario autenticado y envía un mensaje fijo con la librería web-push. No acepta un destinatario arbitrario: cada persona solo puede probar con sus propios dispositivos, lo que evita convertir la prueba en un mecanismo de spam.

El flujo técnico es el siguiente. El navegador pide permiso y, si se concede, obtiene la llave pública VAPID. Con ella crea una suscripción y envía endpoint, p256dh y auth a la API. El servidor los guarda junto al identificador del usuario. Al pulsar «Enviar notificación de prueba», el cliente llama a la ruta protegida; el servidor firma y cifra el mensaje, lo entrega al servicio push y cuenta la respuesta. El service worker recibe el evento, muestra la notificación, avisa a las pestañas abiertas y actualiza la tarjeta y la bitácora. Si el usuario toca el aviso, se abre la página del laboratorio.

8.2 Interfaz del laboratorio

La Figura 1 muestra la columna de operación. El primer bloque informa si el navegador soporta push, cuál es el estado del permiso, si existe una suscripción y si el service worker está activo. El segundo bloque permite enviar la prueba y el tercero registra cada evento con su hora: permiso concedido, suscripción guardada, envíos aceptados y push recibido. Esa bitácora es la evidencia de que el mensaje recorrió el circuito completo.

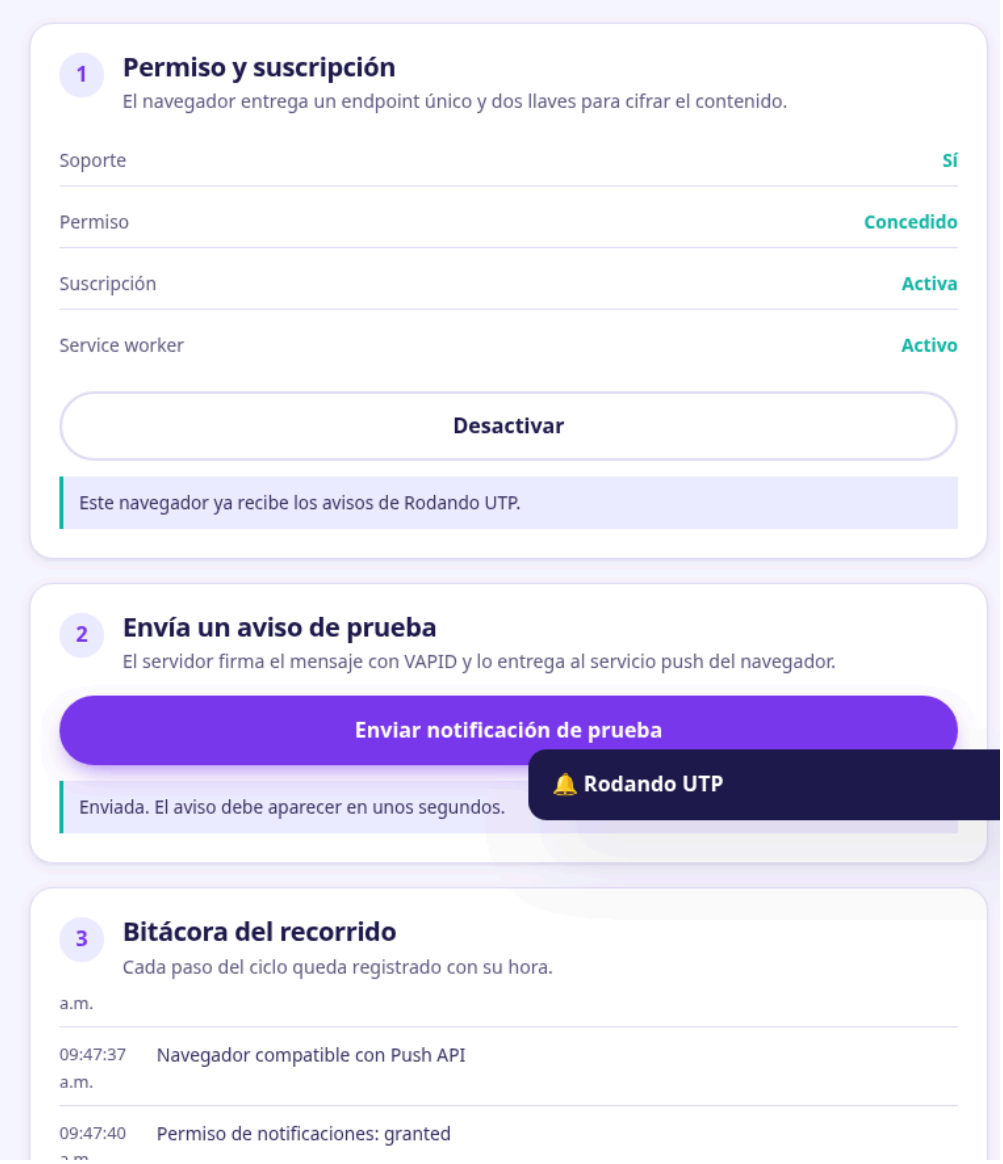


Figura 1. Columna de operación del laboratorio: estado del permiso y la suscripción, envío de la prueba y bitácora con los eventos del ciclo.

A la derecha, la página reserva un espacio para el último aviso recibido y dos bloques explicativos: las tecnologías que intervienen y los usos prácticos. La Figura 2 captura ese costado después de un envío exitoso; la tarjeta superior ya muestra el título y la hora del mensaje, y la bitácora confirma que el servidor aceptó un envío y que el service worker recibió el push.



Figura 2. Aviso recibido y bloques de contexto. La tarjeta y la bitácora cambian cuando el service worker entrega el mensaje a la página.

La vista móvil conserva la misma jerarquía en una columna. La Figura 3 corresponde a una pantalla de 390 píxeles de ancho: el encabezado, el título y el primer bloque se leen sin desplazamiento horizontal y los botones ocupan el ancho disponible. La comprobación midió el ancho del documento y confirmó que no hay desbordamiento.



Figura 3. Vista móvil del laboratorio a 390 píxeles de ancho, sin desbordamiento horizontal.

8.3 Flujo de extremo a extremo

La prueba se ejecutó en un servidor local con HTTPS equivalente (localhost), llaves VAPID de prueba y una base de datos independiente. Se usó Chrome de escritorio con el permiso concedido y una cuenta de estudiante. Los pasos fueron los siguientes:

- Abrir la página y verificar el soporte: Push API, Notifications API y service worker disponibles.
- Pulsar «Activar notificaciones»; el navegador mostró el diálogo y el permiso quedó en «Concedido».
- Crear la suscripción y guardarla en el servidor; la bitácora registró «Suscripción guardada».
- Pulsar «Enviar notificación de prueba»; la API respondió con un envío aceptado sobre una suscripción.
- Esperar la entrega; el service worker mostró el aviso, la tarjeta se actualizó y la bitácora registró «Push recibido».
- Repetir la vista en 390 píxeles y comprobar que el contenido se adapta sin scroll horizontal.

El resultado fue el esperado: el mensaje enviado desde el servidor llegó al navegador sin que la página hiciera una consulta adicional. La evidencia visible es la secuencia de la bitácora, donde aparecen la suscripción, el envío aceptado y la recepción. La notificación del sistema no se incluye como captura porque el entorno de prueba no tenía un servicio de notificaciones de escritorio; la tarjeta de la Figura 2 muestra el mismo contenido que el service worker entregó a la página.

8.4 Resultados de la prueba

Además de la comprobación manual, la suite del backend se amplió con un caso para la ruta de prueba. Las 46 pruebas del proyecto pasan y la revisión de estilo no reporta errores. El caso nuevo verifica dos cosas: que la ruta rechaza una petición sin sesión con 401 y que, cuando el servidor no tiene llaves VAPID, responde 503 con el código PUSH_NO_CONFIGURADO en lugar de fallar de forma indeterminada.

La prueba de entrega real se hizo con las llaves configuradas en el proceso del servidor. La respuesta de la API fue «1 de 1 envíos aceptados» y la página registró el push en el mismo instante. Esa distinción importa: el 503 es el comportamiento esperado cuando falta configuración, no un error del ejemplo. En producción, la función queda activa en cuanto se definen las variables de entorno.

8.5 Configuración y seguridad

Las llaves se generan una vez y se guardan como variables de entorno del servidor: VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY y VAPID_SUBJECT. La llave pública se entrega al navegador; la privada nunca sale del servidor ni se escribe en el repositorio. El sujeto debe ser un correo o una URL de contacto del proyecto, porque el servicio push lo usa para identificar al emisor.

La ruta de prueba exige sesión y limita el envío a las suscripciones del usuario autenticado. El envío administrativo existente conserva su propio control de rol. Cuando el servicio push responde 404 o 410, la suscripción se elimina; así la base no conserva endpoints muertos. El service worker, por su parte, no cachea las rutas de push ni las respuestas con datos personales, y el mensaje que reenvía a las pestañas solo contiene título, texto y URL.

Queda una tarea de operación: configurar las llaves en el entorno publicado. Sin ellas, la página explica que el servidor no está configurado y no intenta simular una entrega. Esa degradación es deliberada: es preferible un mensaje claro a un botón que parece funcionar y no envía nada.

9. Conclusiones

Las notificaciones push cubren el intervalo en el que la aplicación deja de mirarse pero el viaje sigue ocurriendo. En Rodando UTP permiten confirmar una reservación, anunciar que el auto va en camino y recordar la salida sin depender de mensajes externos. La tecnología ya está disponible en la PWA: service worker, HTTPS, manifiesto y una API que puede guardar suscripciones y enviar mensajes firmados.

El ejemplo demuestra el ciclo completo con una interfaz que hace visible cada paso. La bitácora no es un adorno: convierte un proceso invisible en evidencia verificable, desde el permiso hasta la recepción. También muestra los límites con honestidad: sin llaves VAPID no

hay envío, sin permiso no hay aviso y sin conexión no hay entrega. Reconocerlos es parte de diseñar una función confiable.

El siguiente paso es configurar las llaves en el entorno publicado y definir una política de avisos: qué eventos justifican una notificación, con qué anticipación y cómo se puede desactivar. Con esa política, la push deja de ser una capacidad técnica y se convierte en un canal de comunicación útil para estudiantes y conductores.

10. Referencias

MDN Web Docs. (s. f.-a). Notifications API. https://developer.mozilla.org/es/docs/Web/API/Notifications_API

MDN Web Docs. (s. f.-b). Push API. https://developer.mozilla.org/es/docs/Web/API/Push_API

MDN Web Docs. (s. f.-c). PushManager.subscribe(). https://developer.mozilla.org/es/docs/Web/API/PushManager/subscribe

Thomson, M., Damaggio, E., y Raymor, B. (2016). Generic event delivery using HTTP push (RFC 8030). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc8030

Thomson, M., y Beverloo, P. (2017a). Message encryption for web push (RFC 8291). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc8291

Thomson, M., y Beverloo, P. (2017b). Voluntary application server identification (VAPID) for web push (RFC 8292). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc8292

web.dev. (s. f.). Push notifications overview. https://web.dev/articles/push-notifications-overview

WebKit. (2023, 16 de febrero). Web push for web apps on iOS and iPadOS. https://webkit.org/blog/13878/web-push-for-web-apps-on-ios-and-ipados/

web-push-libs. (s. f.). web-push. GitHub. https://github.com/web-push-libs/web-push

Apéndice A. Correspondencia con los requisitos

La Tabla A1 indica dónde se atiende cada punto solicitado en la actividad.

Tabla A1

Correspondencia entre los requisitos y las secciones del reporte

Requisito	Contenido	Sección
Investigar el tema	Definición, propósito y ciclo de funcionamiento	2, 3
Características	Entrega en segundo plano, cifrado, VAPID e integración	4
Ventajas	Comparación con otros canales y beneficios para el proyecto	5
Aplicaciones prácticas	Casos en Rodando UTP y en otros sectores	6
Componentes y tecnologías	Cliente, servidor, protocolos y requisitos	7
Ejemplo funcional	Página, ruta de prueba y flujo de extremo a extremo	8
Capturas	Interfaz de escritorio, aviso recibido y vista móvil	8.2
Conclusiones	Alcance, límites y siguiente paso	9

Apéndice B. Enlaces de la entrega

Aplicación publicada (PWA): https://rodando-utp-m12i.onrender.com

Laboratorio de notificaciones: https://rodando-utp-m12i.onrender.com/notificaciones.html

Código fuente del proyecto: https://github.com/netfoor/pwa-rodando-utp